MainWindow

- escena : QGraphicsScene*
- vista : QGraphicsView*
- timerLoop : QTimer*
- reloj : QElapsedTimer
- config : DifficultyConfig
- engine : GameEngine*
- spriteJugador, spriteLevy, spritePortal, itemFondo : QGraphicsPixmapItem*
- spritesVariantes, spritesPortalesNivel2 : QList<QGraphicsPixmapItem*>
- pixJugadorNormal/NormalIzq/Golpe/GolpeIzq/Walk/WalkIzq : QPixmap
- pixLevyNormal/NormalIzq/Walk/WalkIzq/Golpe/GolpeIzq : QPixmap
- pixVarianteBase/Walk/WalkIzq/Attack/AttackIzq : QPixmap
- pixVariantePortalBase/Walk/WalkIzq/Attack/AttackIzq : QPixmap
- mostrandoGolpe, mirandoDerecha, jugadorMoviendose : bool
- contadorGolpe, contadorAnimacion, contadorLevyGolpe : int
- levyGolpeando : bool- barraVidaJugador, barraVidaLevy, fondoVidaLevy : QGraphicsRectItem*
- barrasVidaEstaticas : QList<QGraphicsRectItem*>
- etiquetasVariantes : QList<QGraphicsTextItem*>
- musicaFondo : QMediaPlayer*
- audioFondo : QAudioOutput*- efectoGolpe,

+ MainWindow(parent : QWidget*)
+ ~MainWindow()
- onUpdate() : void
- procesarInput() - sincronizarSprites()
- actualizarHUD() : void
- iniciarJuego(nivel : int) - cambiarMusica(estadoMusica : int) : void
- dibujarMenuPrincipal()
- dibujarMenuPausa() -dibujarMenuDificultad() : void
- mostrarPantallaFin(victoria : bool)
- reiniciarJuego() : void

<<abstract>> Entidad

x : float
y : float
velX : float
velY : float
vida : float
vidaMaxima : float
masa : float
activo : bool
tiempoRecuperacionGolpe : float

+ Entidad(x, y, vida, masa : float)+ ~Entidad()
+ update(dt : float) : void {abstract}
+ recibirDanio(cantidad : float) : void
+ reducirTiempoRecuperacion(dt : float) : void
# limitarBordes(limX, limY, ancho, alto : float) : void

Jugador

- velBase : float
- cargando : bool
- tiempoCarga : float
- hitStreak : uint8_t
- comboActivo : bool
- invulnerable : bool
- tiempoInvulnerable : float
- danioActual : float«static» MULTIPLICADOR_VELO : float«static» UMBRAL_CARGA : float

+ Jugador(x, y : float)
+ ~Jugador(
+ update(dt : float) : void
+ moverX(dir : float) : void
+ moverY(dir : float) : void
+ detener() : void
+ atacar() : void
+ recibirDanioAmbiental(cantidad : float) : void

Portal

- tiempoVida : float
- tiempoMovimiento : float
- amplitud : float- omegaMAS : float
- radioActual : float- cx : float
- cy : float- radioOrbital : float
- omegaOrbital : float
- fase : float

+ Portal(x, y, amplitud, omegaMAS : float)
+ Portal(cx, cy, radioOrbital, omegaOrbital, fase : float)
+ ~Portal()
+ update(dt : float) : void
+ updateConPhysics(dt : float, ph : PhysicsEngine) : void
+ jugadorTocaBorde(xJ, yJ, ancho, alto : float) : bool

<<abstract>> Enemigo

danio : float
puntosAlDerrotar : uint16_t

+ Enemigo(x, y, vida, masa, danio : float, pts : uint16_t)
+ ~Enemigo()
+ update(dt : float, jug : Jugador) : void {abstract}
+ moverse(dt : float) : void {abstract}
+ recibirDanio(cantidad : float) : void

DifficultyConfig

+ gravedadG : float
+ amplitudMAS : float
+ omegaMAS : float
+ velVarianteBase : float
+ aceleracionVariante : float
+ velMaxVariante : float
+ radioPortalNivel2 : float
+ omegaPortal : float
+ agresividadIA : float
+ velBaseLevy : float
+ cooldownEvasionLevy : float
+ velVoladora : float
+ frecuenciaSpawn : float
+ danioPortal : float
+ danioLevy : float

+ «static» facil() : DifficultyConfig
+ «static» normal() : DifficultyConfig
+ «static» dificil() : DifficultyConfig

GameEngine

- jugador : Jugador*
- enemigos : QList<Enemigo*>
- portales : QList<Portal*>
- physics : PhysicsEngine
- estado : GameState
- config : DifficultyConfig
- tiempoNivel2, tiempoSpawn, tiempoProyectil : float
- nivelActivo : uint8_t
- pausado, creciendoPortales : bool
- targetPortales, totalVariantesSpawneadas : int

+ GameEngine(config : DifficultyConfig)
+ ~GameEngine()
+ iniciarNivel(nivel : uint8_t) : void
+ update(dt : float) : void
+ aplicarAtaqueJugador() : void
+ pausar()
+ reanudar() : void- actualizarNivel1(dt)
- actualizarNivel2(dt : float) : void

PhysicsEngine

-gravedadG : float
- dt : float

+ PhysicsEngine(config : DifficultyConfig)
+ parabolica(x0, y0, v0x, v0y, t : float) : QPointF
+ MAS(A, omega, t : float) : float
+ acelerarClon(velActual, aceleracion, velMax, dt : float) : float
+ reboteBorde(velActual, coefRestitucion : float) : float
+ orbital(cx, cy, r, omega, fase, t : float) : QPointF
+ setDt(dt : float) : void

VariantePortal

- zonaOrigen : int
- impactoAlJugador : bool
«static» pesosZonas : float[9]
«static» zonaActualJugador : int
«static» alpha : float
«static» ANCHO_TERCIO : float
«static» ALTO_TERCIO : float

+ VariantePortal(x, y, vida, masa, danio : float, pts : uint16_t)
+ update(dt : float) : void
+ update(dt : float, jug : Jugador) : void
+ moverse(dt : float) : void
+ setZonaOrigen(zona : int) : void
+ setImpacto(impacto : bool) : void«static»
+ percibir(xJ, yJ : float) : void«static»
+ razonar() : int«static»
+ aprender(zona : int, recompensa : float) : void

Variante

- estado : Estado {RODEAR, ATACAR}
- angulo : float
- tiempoEnEstado : float
- velRodear : float
- velAtacar : float

+ Variante(x, y, vida, masa, danio : float, pts : uint16_t)
+ update(dt : float) : void
+ update(dt : float, jug : Jugador) : void
+ moverse(dt : float) : void
+ setEstado(e : Estado) : void
+setAngulo(a : float) : void
+ setVelocidades(velRodear, velAtacar : float) : void

AngtrongLevy

estadoActual : EstadoIA {AGRESIVO, EVASIVO, DEFENSIVO, GOLPEADO}
- historialDirecciones : deque<int8_t>
- agresividad : float
- radioPortal : float
- velocidadBase : float
- distanciaAlJugador : float
- jugadorCercano : bool
- golpeado : bool- velXGolpe, velYGolpe : float
«static» UMBRAL_DISTANCIA_EVASION : float
«static» UMBRAL_VIDA_DEFENSIVO, UMBRAL_VIDA_AGRESIVO : float
«static» AGRESIVIDAD_MAX, FRECUENCIA_UMBRAL : float
«static» CAPACIDAD_HISTORIAL, INTERVALO_APRENDIZAJE : uint8_t

+ AngstromLevy(x, y : float)
+~AngstromLevy()
+ update(dt : float) : void
+ update(dt : float, jug : Jugador) : void
+ moverse(dt : float) : void
+ recibirImpacto(dirX : float) : void
- percibir(jug : Jugador) - razonar()
- actuar(dt : float)
- aprender() : void

GameState

- estadoPartida : EstadoPartida {MENU, JUGANDO, PAUSA, VICTORIA, DERROTA}
- nivelActual : uint8_t
- puntos : uint32_t
- tiempoRestante : float
- vidaJugador : float
- vidaMaxJugador : float
- vidaLevy : float
- vidaMaxLevy : float- hitStreak : uint8_t
- comboActivo : bool

+ GameState()
+ setEstado(e : EstadoPartida) : void
+ sumarPuntos(cantidad : uint32_t) : void

0..*
0..*